

DATABASE SYSTEMS

CHAP. 8: Concurrency Control Techniques

Concurrency and Concurrency Control

- Concurrency

- Refers to the sharing of resources by multiple interactive users or application programs at the same time.

- Concurrency control

- Concurrency control is a database management systems (DBMS) concept that is used to address conflicts with the simultaneous accessing or altering of data that can occur with a multi-user system.

- Example

- Preventing many passengers from buying the only same last ticket.

Problems due to the Concurrent Execution of Transactions

- The lost update problem
- The incorrect summary or unrepeatable read problem
- The temporary update (dirty read) problem

The lost update problem

- Two transactions accessing the same database item have their operations interleaved in a way that makes the database item incorrect

The incorrect summary or unrepeatable read problem

- One transaction is calculating an aggregate summary function on a number of records while other transactions are updating some of these records.

The temporary update (dirty read) problem

- One transaction updates a database item and then the transaction fails. The updated item is accessed by another transaction before it is changed back to its original value

Methods for Serializability

- The state of the database should always look as if the committed transactions were performed one by one in isolation (i.e., in a serial schedule).

Pessimistic Concurrency Control (or *pessimistic locking*)

- Locks resources as they are required, for the duration of a transaction.

Optimistic Concurrency Control

- Works on the assumption that resource conflicts between multiple users are unlikely ,and allows transactions to execute without locking any resources.

Locking Techniques for Concurrency Control

- A lock is a variable associated with a data item in the database.
- A lock describes the status of the data item with respect to possible operations that can be applied to that item.
- A transaction locks an object before using it
- When an object is locked by another transaction, the requesting transaction must wait

Two-Phase Locking

- *Basic 2pl*
 - When a transaction releases a lock, it may not request another lock
- *Conservative 2pl or static 2pl*
 - A transaction locks all the items it accesses before the transaction begins execution
- *Strict 2pl*
 - A transaction does not release any of its locks until after it commits or aborts

Starvation

- If concurrency control manager is badly designed.
- For example: A transaction may be waiting for an x-lock on an item, while a sequence of other transactions request and are granted an s-lock on the same item. The same transaction is repeatedly rolled back due to deadlocks.